

Movie Analysis USING TMDb REST API

Project Overview

In this project, I used the TMDb REST API to collect and analyze data for about 10,000 movies, each representing a unique user-reviewed title. The dataset includes important features such as:

title — movie name

release_date — official release date

popularity — measure of public interest

vote_average — average user rating (scale 1–10)

vote_count — total number of user ratings

genre_ids — movie genre mappings

Additional metadata like adult content flag and overview

Using Python and Pandas, I cleaned and prepared the data for analysis. The project focuses on visualizing movie trends, analyzing user ratings, and evaluating popularity differences across genres and over time. It demonstrates skills in working with REST APIs, data wrangling on large datasets, and extracting actionable insights through exploratory data analysis.

In [163...

```
#Import Libraries
import requests
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

1. Extract Data using REST API

In [164...

```
import time

api_key = '209d49eb91d0e4da5326c69406fc0f8c'
base_url = "https://api.themoviedb.org/3/movie/popular"

all_movies = []

for page in range(1, 501): # 500 pages x 20 = 10,000 movies
    url = f"{base_url}?api_key={api_key}&language=en-US&page={page}"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        movies = data.get("results", [])
        all_movies.extend(movies)

    time.sleep(0.3) # ~3 API calls per second (safe under TMDb's rate limit)

# Convert to DataFrame
df = pd.DataFrame(all_movies)

# Save to CSV
df.to_csv("tmdb_popular_movies.csv", index=False)

print(f"\n Collected {len(df)} movies in total!")
```

Collected 9996 movies in total!

2. Explore, Clean and Transform Data

In [165...

```
api_key = '209d49eb91d0e4da5326c69406fc0f8c'

genre_url = f"https://api.themoviedb.org/3/genre/movie/list?api_key={api_key}&language=en-US"

response = requests.get(genre_url)
genres_data = response.json()

# Create a dictionary that maps genre ID to name
genre_dict = {genre['id']: genre['name'] for genre in genres_data['genres']}
```

In [166... `df.head()`

Out[166...

	adult	backdrop_path	genre_ids	id	original_language	original_title	overview	popularity	
0	False	/7Q2CmqIVJuDAESPPp76rWliA0AD.jpg	[28, 12, 18]	1011477	en	Karate Kid: Legends	After a family tragedy, kung fu prodigy Li Fon...	629.7943	/
1	False	/ovZasZ9EeZcp6UsrElkQ63hFCd.jpg	[14, 10751, 28]	1087192	en	How to Train Your Dragon	On the rugged isle of Berk, where Vikings and ...	558.2017	/q5p>
2	False	/ApRxyHFuvv5yghedxXPJSm9FEDe.jpg	[878, 12, 28]	1061474	en	Superman	Superman, a journalist in Metropolis, embarks ...	507.0710	/omb
3	False	/qtSY2SAL5QApuCUD0sXqyzgHYnl.jpg	[28, 35, 80, 9648]	1374534	nl	Bad Boa's	When an overeager community officer and a reck...	495.8851	,
4	False	/5l3Ge1f3944AsDiiDXWQ3RSZ0Bg.jpg	[28, 878, 53]	1071585	en	M3GAN 2.0	After the underlying tech for M3GAN is stolen ...	390.3905	/4i

In [167... `df.shape`

Out[167... (9996, 14)

9,996 rows and 14 columns.

In [168... `df.dtypes`

```
Out[168...  adult                bool
  backdrop_path        object
  genre_ids            object
  id                   int64
  original_language    object
  original_title       object
  overview            object
  popularity          float64
  poster_path         object
  release_date        object
  title               object
  video               bool
  vote_average        float64
  vote_count          int64
  dtype: object
```

In [169... `df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9996 entries, 0 to 9995
Data columns (total 14 columns):
#   Column                Non-Null Count  Dtype
---  -
0   adult                 9996 non-null   bool
1   backdrop_path         9771 non-null   object
2   genre_ids             9996 non-null   object
3   id                   9996 non-null   int64
4   original_language     9996 non-null   object
5   original_title        9996 non-null   object
6   overview              9996 non-null   object
7   popularity            9996 non-null   float64
8   poster_path          9967 non-null   object
9   release_date          9996 non-null   object
10  title                 9996 non-null   object
11  video                 9996 non-null   bool
12  vote_average          9996 non-null   float64
13  vote_count            9996 non-null   int64
dtypes: bool(2), float64(2), int64(2), object(8)
memory usage: 956.8+ KB
```

```
In [170... def map_genres(genre_id_list):
               return [genre_dict.get(genre_id, "Unknown") for genre_id in genre_id_list]

df['genre_names'] = df['genre_ids'].apply(map_genres)
```

```
In [171... #Check that genre_names column worked correctly
df.head(3)
```

Out[171...

	adult	backdrop_path	genre_ids	id	original_language	original_title	overview	popularity	
0	False	/7Q2CmqIVJuDAESPPp76rWliA0AD.jpg	[28, 12, 18]	1011477	en	Karate Kid: Legends	After a family tragedy, kung fu prodigy Li Fon...	629.7943	/
1	False	/ovZasZ9EeZcp6UsrElkQ63hFCd.jpg	[14, 10751, 28]	1087192	en	How to Train Your Dragon	On the rugged isle of Berk, where Vikings and ...	558.2017	/q5p>
2	False	/ApRxyHFuvv5yghedxXPJSm9FEDe.jpg	[878, 12, 28]	1061474	en	Superman	Superman, a journalist in Metropolis, embarks ...	507.0710	/omb

In [172...

```
#Drop Unwanted Columns
df.drop(['backdrop_path', 'genre_ids', 'original_title', 'adult', 'poster_path', 'video'], axis=1, inplace = True)
```

In [173...

```
df.head()
```

Out[173...

	id	original_language	overview	popularity	release_date	title	vote_average	vote_count	genre_names
0	1011477	en	After a family tragedy, kung fu prodigy Li Fon...	629.7943	2025-05-08	Karate Kid: Legends	7.304	415	[Action, Adventure, Drama]
1	1087192	en	On the rugged isle of Berk, where Vikings and ...	558.2017	2025-06-06	How to Train Your Dragon	8.031	735	[Fantasy, Family, Action]
2	1061474	en	Superman, a journalist in Metropolis, embarks ...	507.0710	2025-07-09	Superman	7.500	709	[Science Fiction, Adventure, Action]
3	1374534	nl	When an overeager community officer and a reck...	495.8851	2025-07-10	Almost Cops	5.868	57	[Action, Comedy, Crime, Mystery]
4	1071585	en	After the underlying tech for M3GAN is stolen ...	390.3905	2025-06-25	M3GAN 2.0	7.700	191	[Action, Science Fiction, Thriller]

In [174...

```
#Change datatype of 'release_date' to Datetime
df['release_date'] = pd.to_datetime(df['release_date'], errors='coerce')
```

In [175...

```
df.dtypes
```

```
Out[175... id                int64
original_language  object
overview          object
popularity        float64
release_date      datetime64[ns]
title            object
vote_average      float64
vote_count        int64
genre_names       object
dtype: object
```

```
In [176... #Reorder Columns
df = df[['id','title','release_date','popularity','vote_average','vote_count','genre_names','overview','original_lang
```

```
In [177... df.head()
```

Out[177...

	id	title	release_date	popularity	vote_average	vote_count	genre_names	overview	original_language
0	1011477	Karate Kid: Legends	2025-05-08	629.7943	7.304	415	[Action, Adventure, Drama]	After a family tragedy, kung fu prodigy Li Fon...	en
1	1087192	How to Train Your Dragon	2025-06-06	558.2017	8.031	735	[Fantasy, Family, Action]	On the rugged isle of Berk, where Vikings and ...	en
2	1061474	Superman	2025-07-09	507.0710	7.500	709	[Science Fiction, Adventure, Action]	Superman, a journalist in Metropolis, embarks ...	en
3	1374534	Almost Cops	2025-07-10	495.8851	5.868	57	[Action, Comedy, Crime, Mystery]	When an overeager community officer and a reck...	nl
4	1071585	M3GAN 2.0	2025-06-25	390.3905	7.700	191	[Action, Science Fiction, Thriller]	After the underlying tech for M3GAN is stolen ...	en

3. Data Analysis and Visualization

In [178...

```

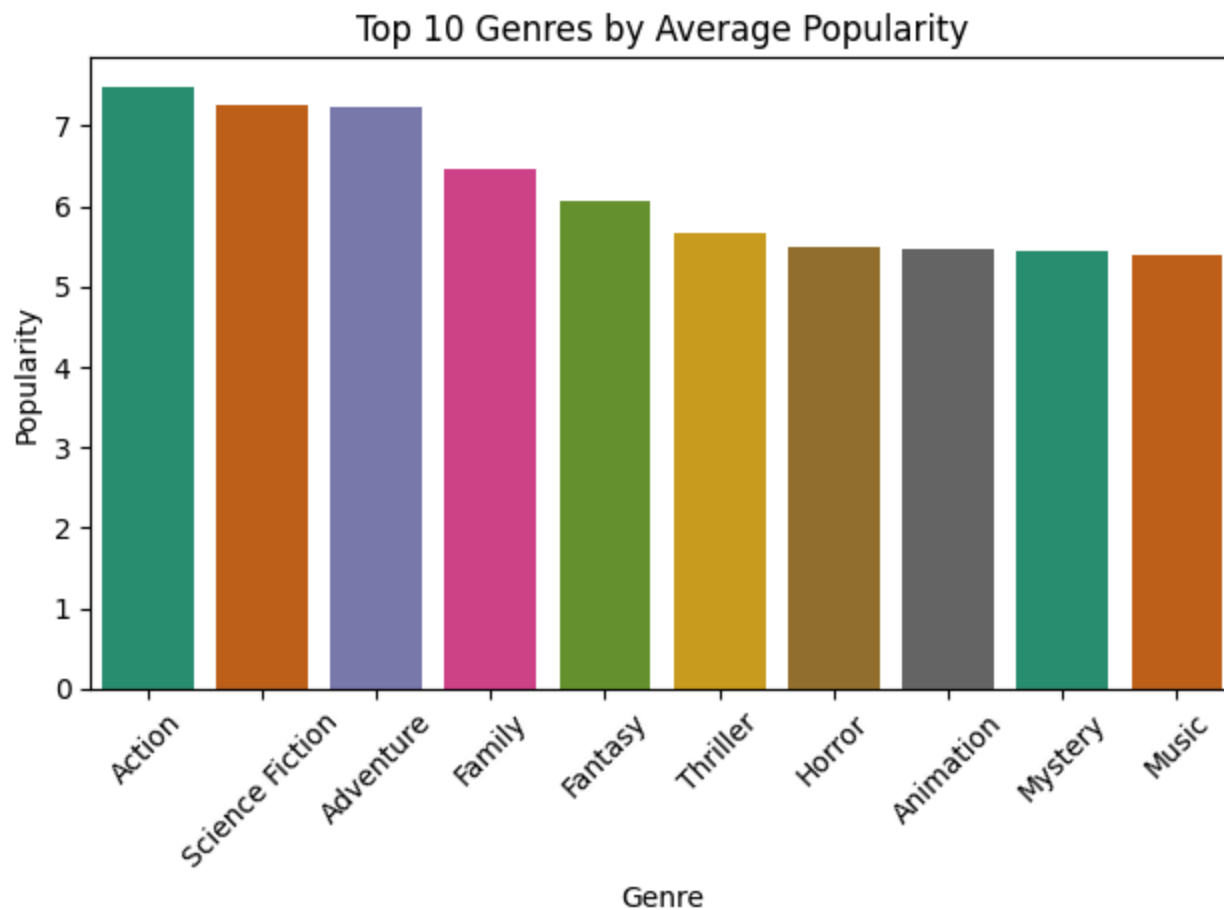
#Explode list of genres
df_exploded = df.explode('genre_names')
top_genres = df_exploded.groupby('genre_names')['popularity'].mean().sort_values(ascending=False).head(10)

# Convert to DataFrame for Seaborn
top_genres_df = top_genres.reset_index()
top_genres_df.columns = ['genre_names', 'popularity']

# Plot Bar Chart using Seaborn
sns.barplot(data=top_genres_df, x='genre_names', y='popularity', hue='genre_names', dodge=False, palette='Dark2')

```

```
plt.title('Top 10 Genres by Average Popularity')  
plt.ylabel("Popularity")  
plt.xlabel("Genre")  
plt.xticks(rotation=45)  
plt.tight_layout()  
plt.show()
```



This bar chart shows the average popularity of the top 10 genres. Action and Adventure genres are the most popular on average, while Mystery, Horror, Animation, and Music have the lowest popularity scores among the top genres.

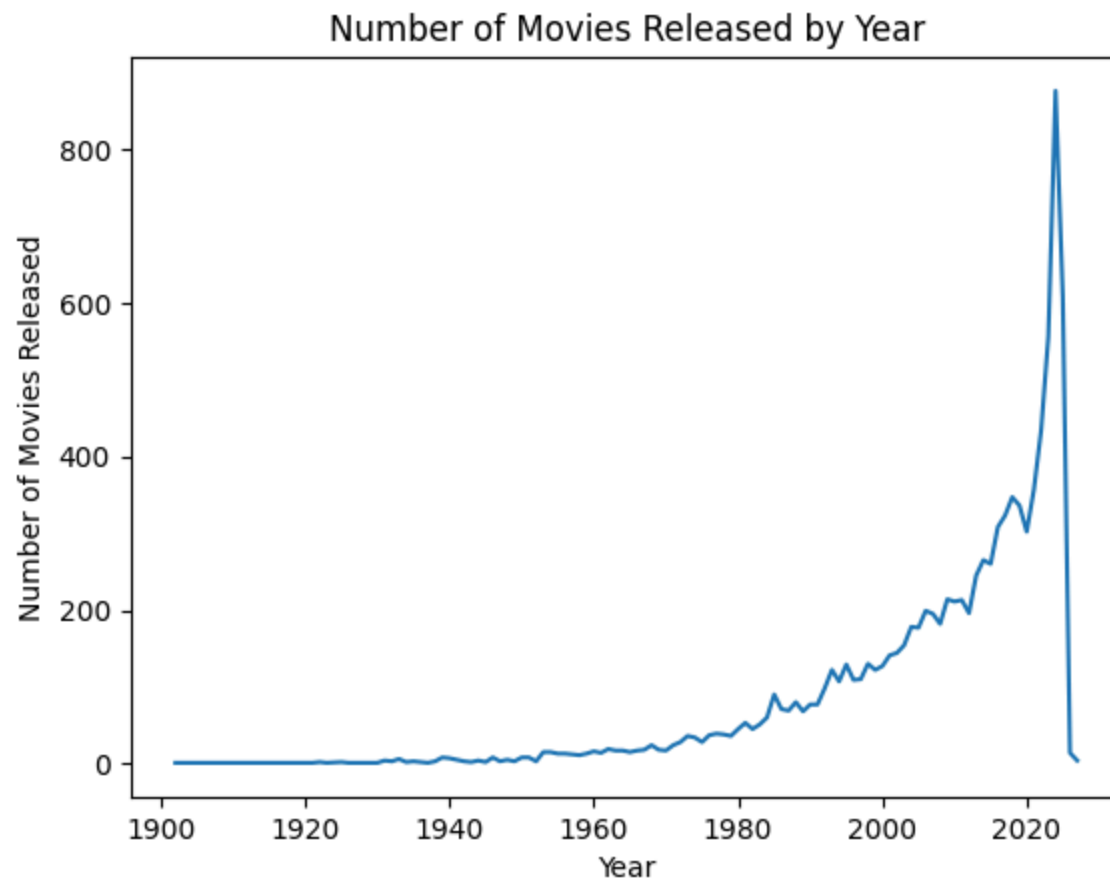
```
In [179... #Add release_year column  
df['release_year'] = df['release_date'].dt.year
```

```
In [180... #Create dataframe and plot Line Chart  
movies_per_year = df.groupby('release_year').size().reset_index(name='movie_count')  
sns.lineplot(data = movies_per_year,x='release_year',y='movie_count', palette='Dark2')  
plt.title('Number of Movies Released by Year')  
plt.xlabel('Year')  
plt.ylabel('Number of Movies Released')
```

C:\Users\zaque\AppData\Local\Temp\ipykernel_13668\1047439171.py:3: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.

```
sns.lineplot(data = movies_per_year,x='release_year',y='movie_count', palette='Dark2')
```

```
Out[180... Text(0, 0.5, 'Number of Movies Released')
```



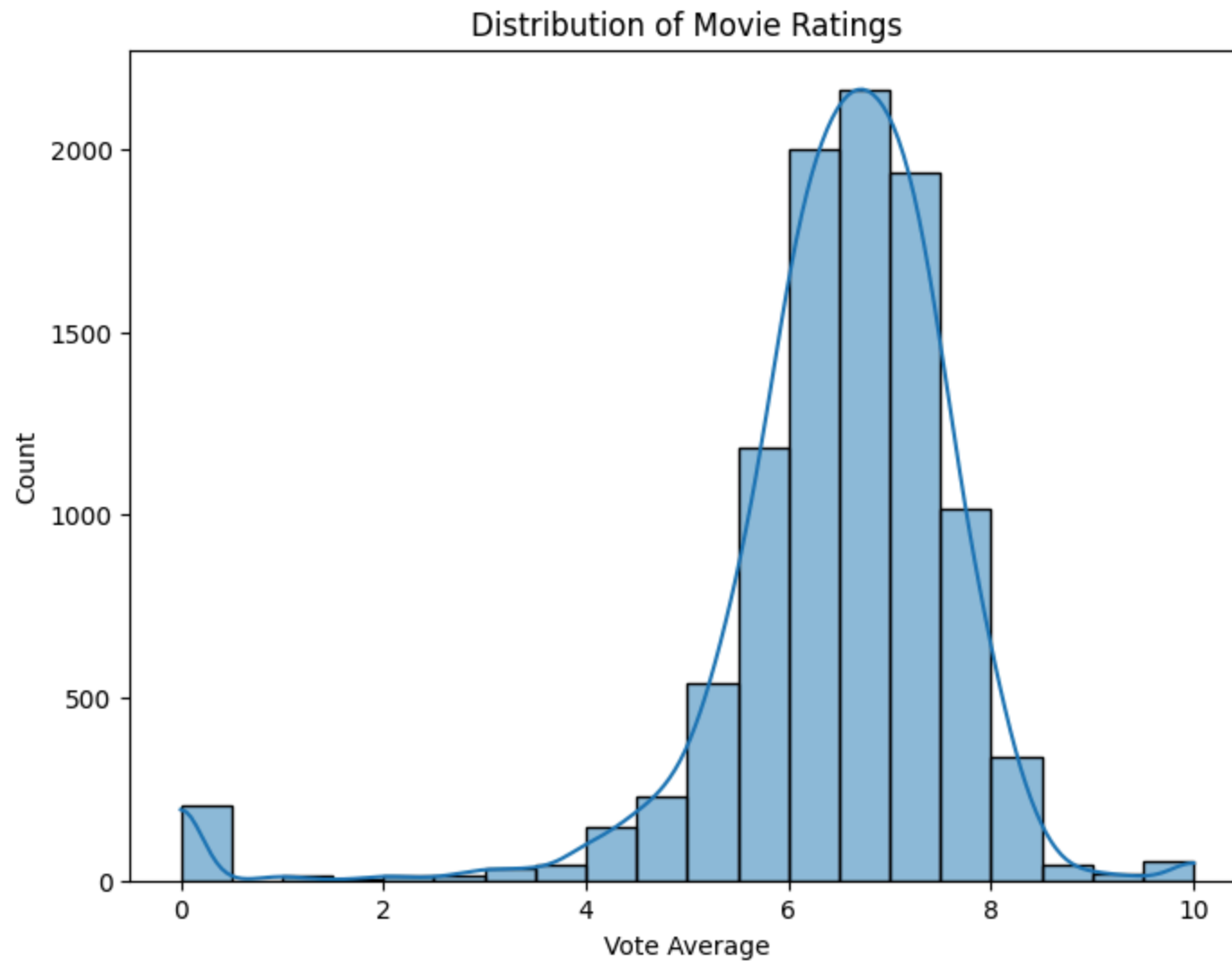
The line chart shows the number of movies released per year. There's a slow but steady increase until around 2020, after which the number of releases grows rapidly — suggesting a surge in movie production in recent years.

In [181...

```
#Plot Histogram of movie ratings
plt.figure(figsize=(8,6))
sns.histplot(df['vote_average'], bins=20, kde=True,palette='Dark2')
plt.xlabel('Vote Average')
plt.title('Distribution of Movie Ratings')
plt.show()
```

```
C:\Users\zaque\AppData\Local\Temp\ipykernel_13668\1930599218.py:3: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
```

```
sns.histplot(df['vote_average'], bins=20, kde=True, palette='Dark2')
```



This histogram shows the distribution of movie ratings. Most movies have ratings clustered around 7, with fewer movies rated below 4 or above 8, indicating a concentration of average to above-average ratings.

4. Key Insights and Conclusion

Overall, this analysis reveals several key patterns in the movie dataset. First, genres like Action and Adventure consistently perform well in terms of popularity, whereas genres such as Mystery, Horror, and Music tend to have lower average popularity. Second, movie production has increased sharply in the last decade, especially around 2020, pointing to a boom likely driven by digital platforms and independent production. Lastly, the distribution of movie ratings is centered around 7, with relatively few extreme values, suggesting that most movies receive moderate critical reception. These insights can help inform decisions about genre focus, production trends, and rating expectations in the film industry.